

Player.tsx — Documentacao Comentada (Karaoke CT)

Data: 16 de Maio de 2026

Visao Geral

O componente Player e a tela principal de reproducao de musicas do Karaoke CT. Ele combina um player de video (Bunny Stream ou arquivos locais), uma barra de busca para adicionar musicas a fila, uma tela de pontuacao (karaoke score), um painel de fila de espera e um QR Code para controle remoto via celular.

Dependencias

React e hooks: useState, useCallback, useMemo, useEffect, useRef

Roteamento: wouter (useParams, Link, useLocation)

Icones: lucide-react (ArrowLeft, Play, Music, etc.)

Hooks do projeto: useDebounce, useToast, useLocalMusic, useQueue, useSession

Componentes UI: shadcn/ui (Skeleton, Button, Input)

API gerada: Orval (useGetMusica, useSearchMusicas)

Dialog: AddToQueueDialog (nome do cantor)

QR Code: react-qr-code

Funcoes Utilitarias

randomScore()

```
function randomScore(): number {  
  return Math.floor(Math.random() * 21) + 80;  
}
```

Gera uma pontuacao aleatoria entre 80 e 100. Math.random() gera 0 a 0.999, multiplica por 21 (0-20.999), Math.floor arredonda (0-20), soma 80 (resultado: 80-100).

Componentes Filhos

PersistentSearchBar

Barra de busca fixa no header. Permite buscar musicas por artista, titulo ou codigo. Usa debounce de 300ms. Fecha dropdown ao clicar fora (nao usa onBlur). Adiciona a fila com dialog para nome do cantor.

Fluxo: Usuario clica "+" -> openQueueDialog() verifica fila < 30 -> verifica duplicata -> abre AddToQueueDialog -> usuario digita nome -> handleConfirmQueue() adiciona via addToQueue() -> toast de confirmacao.

ScoreScreen

Tela de pontuacao apos o video terminar. Mostra pontuacao, estrelas, rotulo e proxima musica.

Fluxo: Pontuacao 100=5 estrelas "Perfeito!", 95-99=5 estrelas "Incrivel!", 90-94=4 estrelas "Excelente!", 85-89=3 estrelas "Muito Bom!", 80-84=2 estrelas "Bom trabalho!". SVG circular com stroke-dasharray. Enter avanca para proxima musica.

QueuePanel

Painel da fila de musicas. Mostra ordem de execucao, permite remover individual ou limpar tudo.

Fluxo: Mostra posicao, icone cantor, nome, musica-artista, codigo #id, botao remover. Maximo 30 musicas. Dados persistem em localStorage via QueueProvider.

Componente Principal: Player (export default)

Estados Principais

Estado	Tipo	Descricao
score	number null	Pontuacao atual (null = nao mostrar)
videoKey	number	Chave para forcar recarregamento do video
panel	"none" "search" "queue"	Painel ativo (fila ou busca)
sessionId	string null	ID da sessao compartilhada para QR Code
nextQueuedItem	QueueItem null	Proxima musica (para tela de pontuacao)

Ciclo de Vida

1. Montagem (useEffect)

Tenta reutilizar sessao salva no localStorage. Se nao existir, cria nova. A sessao permite que celulares controlem a fila via /remote/{id}.

2. Fim do Video (handleVideoEnd)

Captura proximo item da fila. Gera pontuacao aleatoria. Mostra ScoreScreen.

3. Proxima Musica (handleNext)

Remove primeira musica da fila (shiftQueue). Incrementa videoKey. Navega para /player/{id}.

4. Repetir (handleReplay)

Limpa pontuacao. Incrementa videoKey (reinicia video atual).

Renderizacao do Video

O player tenta reproduzir em 3 modos, em ordem de prioridade:

1. Bunny Stream (online)

Requer VITE_BUNNY_LIBRARY_ID. Usa iframe do Bunny Stream com autoplay.

2. Arquivo Local (offline)

Usa useLocalMusic() para pasta de MP4. Usa tag <video> nativa HTML5.

3. Tela de erro

Mostra mensagem se arquivo nao encontrado. Permite selecionar pasta.

QR Code (Controle Remoto)

Sessao criada automaticamente ao abrir o player.
URL gerada: {origem}/remote/{sessionId}
QR Code renderizado no canto inferior direito do video. Usuario escaneia com celular e abre interface /remote/{id} para adicionar musicas. Fundo amarelo/primary translucido (bg-primary/80) para contrastar com o video.

Integracao com Contextos

useQueue (Fila Local)

- queue: Array de musicas (max 30)
- addToQueue(): Adiciona ao final
- removeFromQueue(): Remove por ID
- shiftQueue(): Remove e retorna primeira
- clearQueue(): Limpa tudo
- isInQueue(): Verifica duplicata

useSession (Sessao Compartilhada)

- createSession(): Cria sessao com ID aleatorio
- joinSession(): Entra em sessao existente
- Sincroniza fila entre multiplos dispositivos

useLocalMusic (Arquivos Locais)

- folderName: Nome da pasta selecionada
- selectFolder(): Abre dialog para escolher pasta
- getFileUrl(): Retorna URL do MP4 pelo ID

Fluxo Completo de Uso

1. Usuario busca musica na barra de busca do header
2. Clica "+" para adicionar a fila
3. Digita o nome do cantor no dialog
4. Musica aparece na fila (painel "Fila")
5. Toca a musica atual (video em tela cheia)
6. Ao terminar, aparece tela de pontuacao
7. Opcional: clicar "Comecar" para proxima da fila
8. Outros podem adicionar musicas via QR Code no celular

Arquitetura de Componentes

```
Player (pagina principal)
|-- Header
|   |-- Botao Voltar
|   |-- Titulo da musica (escondido em mobile)
|   |-- PersistentSearchBar
|   |   |-- Campo de busca
|   |   |-- Dropdown de resultados
|   |   |-- AddToQueueDialog (modal nome do cantor)
|   |-- Botao Pontuacao
|   |-- Botao Fila (com contador)
|-- QueuePanel (condicional)
|   |-- Lista de musicas na fila
|-- Video Area
|   |-- Player Bunny Stream (iframe)
|   |-- Player Local (video tag)
|   |-- ScoreScreen (overlay)
|   |-- QR Code (canto inferior direito)
|-- Gradient bottom (decorativo)
```

Consideracoes Tecnicas

1. Stale closures: Usa useRef para guardar referencias atualizadas de shiftQueue e navigate, evitando problemas com closures em callbacks memorizados.
2. SSR-safe: typeof window !== "undefined" antes de acessar window.location para evitar erros em server-side rendering.
3. Debounced search: 300ms de debounce para evitar requisicoes excessivas a API enquanto o usuario digita.
4. Click-outside: Listener de mousedown no documento para fechar dropdown de busca ao clicar fora, sem depender do onBlur que quebrava cliques nos botoes.